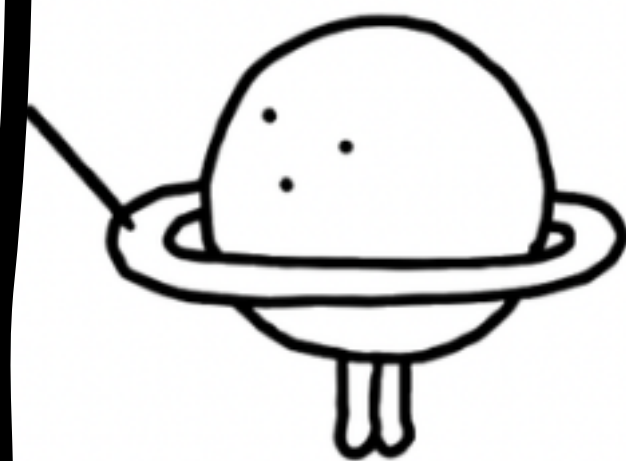
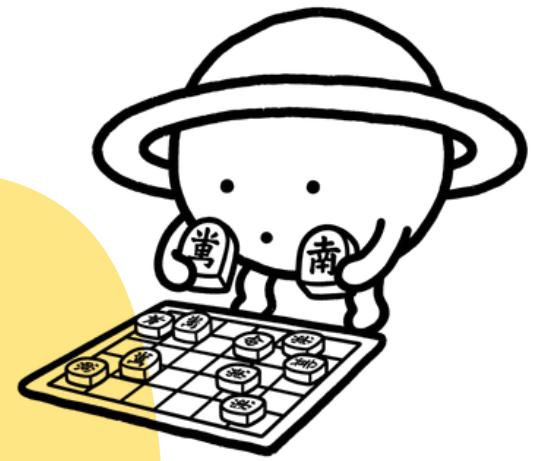


그 추상화, 정말 필요한가요?

장기 기물 설계로 돌아본 추상화의 비용



목차



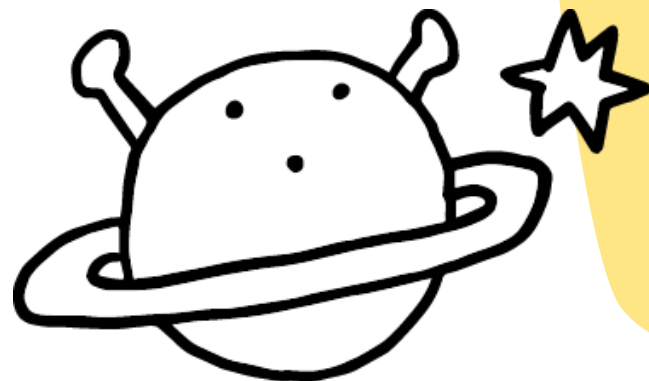
1. 들어가며: 추상화가 필요한 이유

2. 장기 기물 추상화: 상속 vs 조합

3. MoveStrategy: 어디까지 분리
할 것인가?

4. 마치며: 과도한 추상화가 아닌

직관적인 코드를 향해



1 들어가며: 추상화가 필요한 이유

장기 기물마다 이동 규칙이 다르다

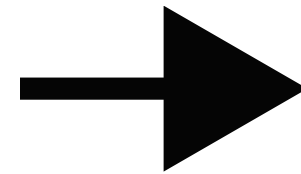
차:
직선 이동

졸/병:
후진 불가

포:
뛰어넘기

궁/사
한칸 이동

마/상:
대각선 이동



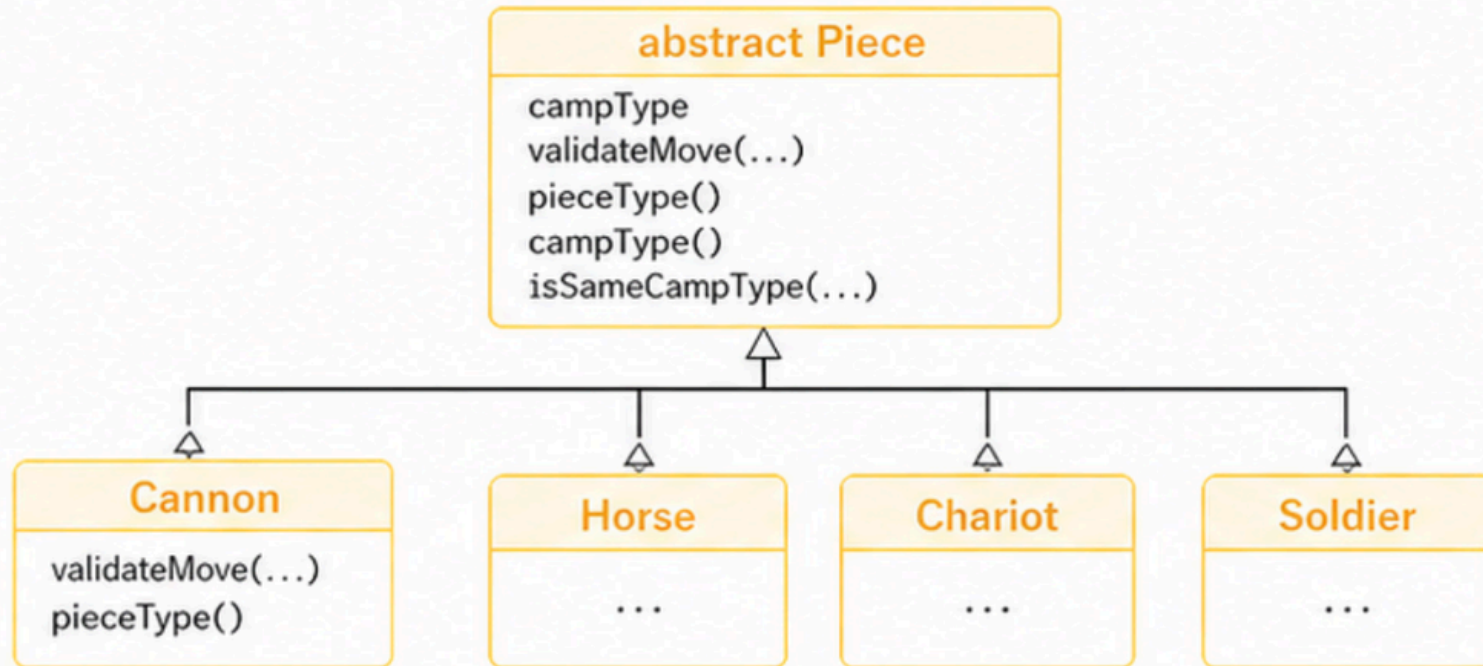
조건문으로 구현하면?

```
if (piece == 차) ...  
else if (piece == 포) ...  
else if (piece == 마) ...  
else if (piece == 상) ...
```

> 변경이 코드 전반으로 퍼지고, 유지보수 비용 증가

2 장기 기물 추상화: 상속 vs 조합

Piece 상속 기반 설계



```
public abstract class Piece {
    private final CampType campType;

    protected Piece(CampType campType) {
        this.campType = campType;
    }

    public abstract void validateMove(Position source,
        Position destination, BoardChecker board);

    public abstract PieceType pieceType();

    public final CampType campType() {
        return campType;
    }

    public final boolean isSameCampType(CampType other) {
        return this.campType == other;
    }
}
```

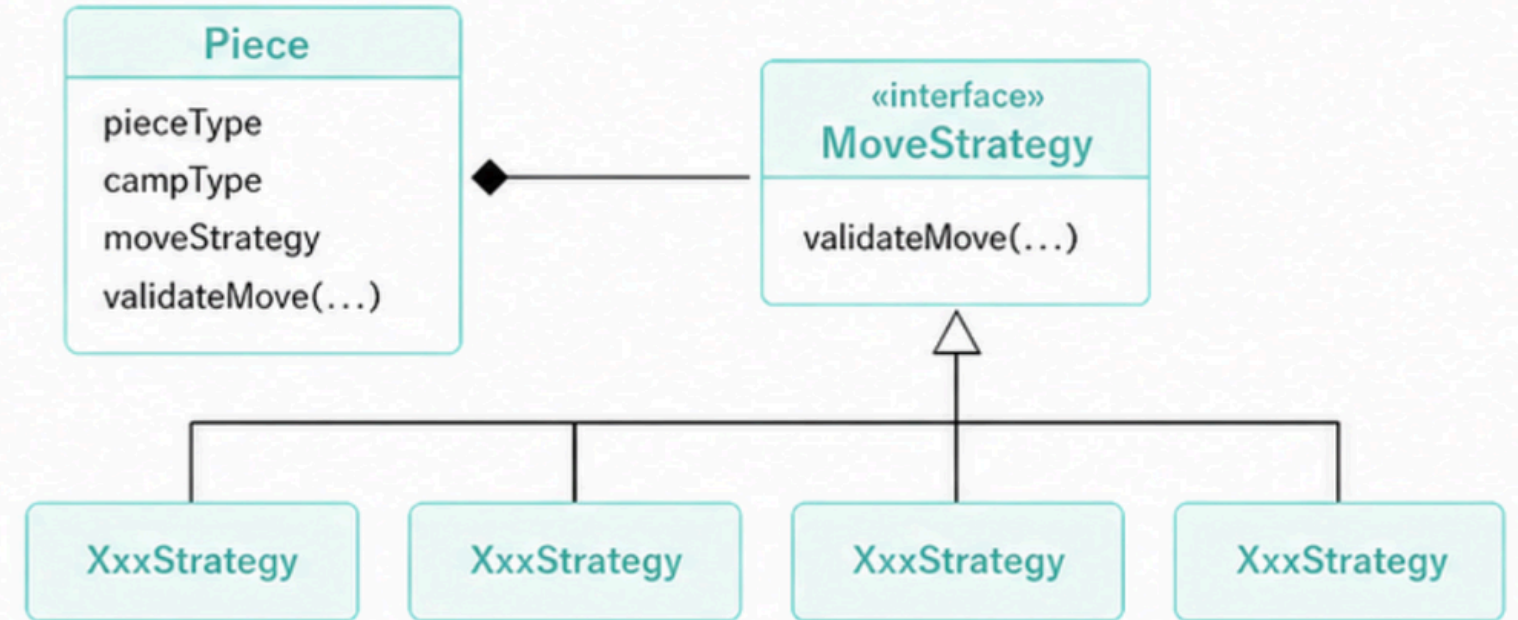
```
public class Cannon extends Piece {
    public Cannon(CampType campType) {
        super(campType);
    }

    @Override
    public void validateMove(Position source,
        Position destination, BoardChecker board) {
        // 포 이동 규칙 검증
    }

    @Override
    public PieceType pieceType() {
        return PieceType.CANNON;
    }
}
```

VS

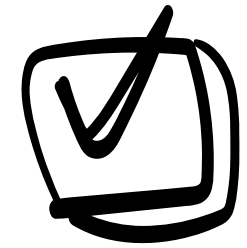
MoveStrategy 기반 설계



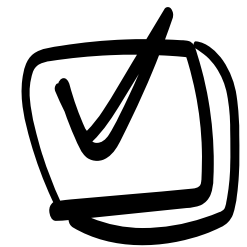
```
public record Piece(PieceType pieceType, CampType campType) {

    public void validateMove(Position source, Position destination, BoardChecker board) {
        pieceRule.validateMove(source, destination, board);
    }
}
```

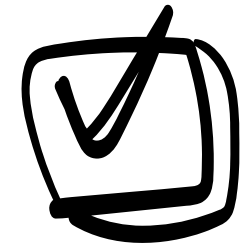
② 장기 기물 추상화: 조합을 선택한 이유



변하는 것은 기물이 아닌 **이동 규칙**



상속은 **부모 클래스까지 코드를 추적**해야
하므로 코드 이해가 어려움



높은 결합도로 인한 낮은 유연성

3 MoveStrategy: 어디까지 분리할 것인가?



첫 번째 시도: 공통 로직 일반화

```
GENERAL(new SingleStepStrategy(), new EmptyCondition()),  
CHARIOT(new MultiStepStrategy(), new EmptyCondition()),  
HORSE(new HorseStrategy(), new EmptyCondition()),  
CANNON(new MultiStepStrategy(), new OnePieceExistsCondition()),  
GUARD(new SingleStepStrategy(), new EmptyCondition()),  
ELEPHANT(new ElephantStrategy(), new EmptyCondition()),  
SOLDIER(new SoldierStrategy(), new EmptyCondition());
```

③ MoveStrategy: 어디까지 분리할 것인가?

Strategy와 연관된 테스트코드들을 계속 살펴보니 추상화 수준이 다들 다른 것 같습니다. 상, 마, 병에 대한 높은 추상화 수준의 Strategy와 추상화 수준이 조금 낮은 단순 직선 혹은 한칸 이동에 대한 Strategy가 같은 추상화 수준으로 다루다보니 어색한 생각이 드네요. Strategy와 연관된 추상화 수준을 동일하게 맞추는 것도 좋을 것 같아요~

> 전략들의 추상화 수준이 동일하지 않음



Q. 추상화 레벨이 맞는 것은 왜 중요할까?

**A. 동료와 같은 기준으로 코드를 이해하고,
확장 방향을 결정할 수 있기 때문**



SingleStrategy

이동 방식 단위



HorseStrategy

기물 단위

> 규칙 추가/수정 시:

어떤 기준으로 코드를 나눠야 할지 판단하기 어려워짐

3 MoveStrategy: 어디까지 분리할 것인가?



두 번째 시도: 공통 로직 추가 분리

```
GENERAL(new SingleStepRule(false)),  
CHARIOT(new EmptySlidingRule()),  
HORSE(new DiagonalStepRule(1)), // 대각선 방향으로 몇 칸 이동하는지  
CANNON(new JumpingSlidingRule()),  
GUARD(new SingleStepRule(false)),  
ELEPHANT(new DiagonalStepRule(2)),  
SOLDIER(new SingleStepRule(true)); // 후진 이동 불가 true
```

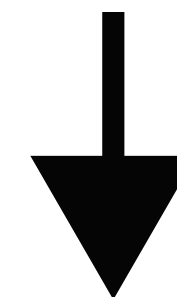
③ MoveStrategy: 어디까지 분리할 것인가?

> 지금 공통인 부분이, 나중에도 같은 변경 단위일까?

③ MoveStrategy: 어디까지 분리할 것인가?

```
public class DiagonalStepRule extends BaseMoveRule {  
  
    private static final int STRAIGHT_DISTANCE = 1;  
  
    private final int diagonalDistance;  
  
    public DiagonalStepRule(int diagonalDistance) {  
        this.diagonalDistance = diagonalDistance;  
    }  
  
    @Override  
    public void validate(Position source, Position destination, Camp camp, BoardChecker board, PieceRule pieceRule) {  
        DirectionInformation direction = new DirectionInformation(source, destination);  
        validateDistance(direction);  
        List<Position> path = findPath(source, direction);  
        validateEmptyPath(path, board);  
    }  
  
    private void validateDistance(DirectionInformation direction) {  
        if (!direction.hasAbsDifferences(diagonalDistance, STRAIGHT_DISTANCE + diagonalDistance)) {  
            throw new IllegalArgumentException(  
                ExceptionMessage.INVALID_DIAGONAL_STEP_MOVE.getMessage(STRAIGHT_DISTANCE, diagonalDistance)  
            );  
        }  
    }  
}
```

상만
대각선 이동 후
직진으로 변경



전략을 함께
사용하는
다른 기물에 영향을 줌

③ MoveStrategy: 어디까지 분리할 것인가?



세 번째 시도: 모든 규칙 분리하기

```
GENERAL(new SingleStepRule()),  
HORSE(new SingleStepRule(1), new DiagonalStepRule(1)),  
CANNON(new DifferentPieceTypeRule(), new SlidingRule(),  
        new OnePieceExistsRule(), ...),
```

-
-
-

③ MoveStrategy: 어디까지 분리할 것인가?



> 그래서 ‘포’가 어떻게 움직인다는 거지?

‘포’를 이해하기 위해
분산된 여러 전략 클래스를 살펴봐야 한다.

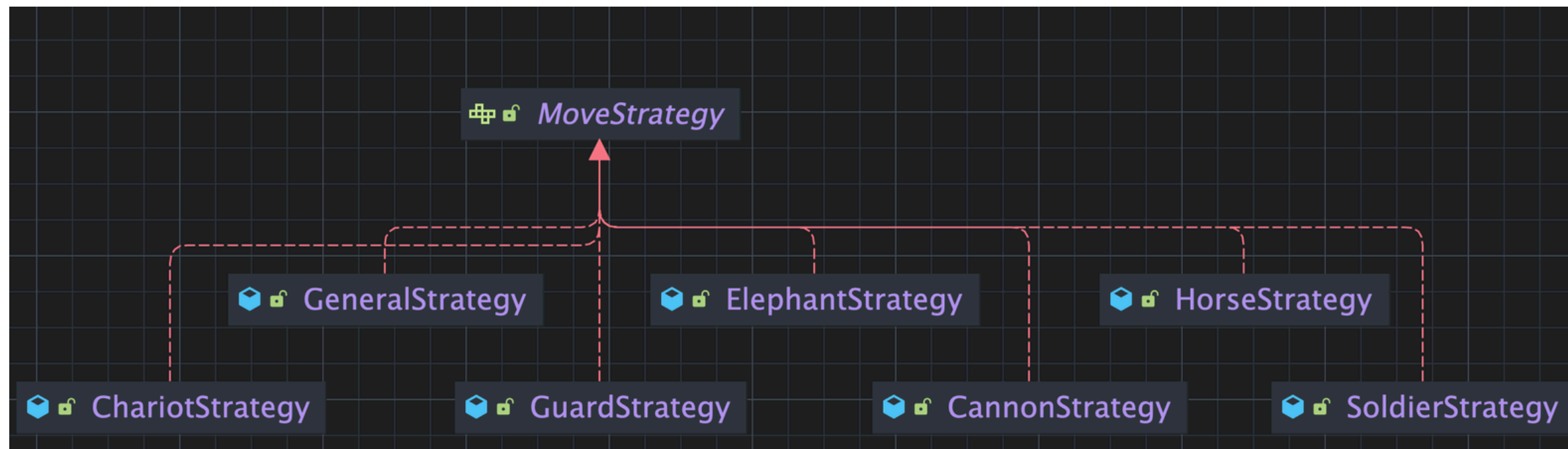
③ MoveStrategy: 어디까지 분리할 것인가?



최종: 기물 별 분리하기

```
GENERAL(new GeneralStrategy()),  
CHARIOT(new ChariotStrategy()),  
HORSE(new HorseStrategy()),  
CANNON(new CannonStrategy()),  
GUARD(new GuardStrategy()),  
ELEPHANT(new ElephantStrategy()),  
SOLDIER(new SoldierStrategy());
```

3 MoveStrategy: 어디까지 분리할 것인가?



기물 이동 규칙이
하나의 클래스에
응집됨

단순한 구조

코드 읽는 비용
감소

4 마침며: 과도한 추상화가 아닌 직관적인 코드를 향해



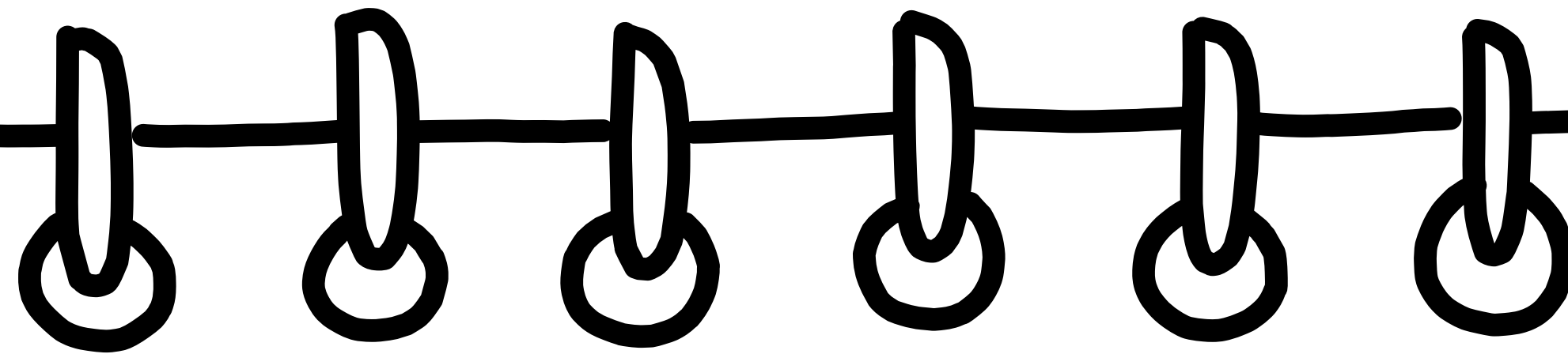
YAGNI

완벽한 설계를
처음부터 만들기보다,
요구사항의 변화에 맞춰
점진적으로 구조를
발전시키는 방향

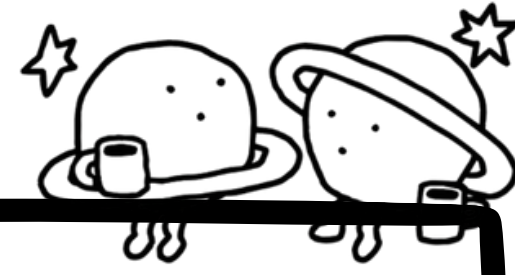


읽기 쉬운 코드

동료가 쉽게
이해하고
수정할 수 있는 코드



**모든 공통점을
추상화 하기보다,
단순하게 남겨두는 선택도
필요하지 않을까요?**



끗

감사합니다